

Module 23.11: What is Gradient Descent in Linear Regression?

1. The Core Optimization Principle

Gradient Descent is a general first-order iterative optimization algorithm used to find the local minimum of a differentiable function. In linear regression, since the MSE cost function $J(w, b)$ is strictly convex, gradient descent is guaranteed to converge to the unique global minimum (given an appropriate learning rate).

```

Cost J(w, b)
  \ <-- Starting point: high cost, steep gradient
  \
  \ Step: w := w - alpha * (dJ/dw)
  \
  \_____<-- Convergence: gradient approx 0
  
```

2. Derivation of Partial Derivatives (Gradients)

Recall the cost function:

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m \left(wx^{(i)} + b - y^{(i)} \right)^2$$

Gradient with Respect to Weight w :

Applying the chain rule:

$$\frac{\partial J}{\partial w} = \frac{1}{2m} \sum_{i=1}^m 2 \left(wx^{(i)} + b - y^{(i)} \right) \cdot \frac{\partial}{\partial w} \left(wx^{(i)} + b - y^{(i)} \right)$$

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right) x^{(i)}$$

Gradient with Respect to Bias b :

Applying the chain rule:

$$\frac{\partial J}{\partial b} = \frac{1}{2m} \sum_{i=1}^m 2 \left(wx^{(i)} + b - y^{(i)} \right) \cdot \frac{\partial}{\partial b} \left(wx^{(i)} + b - y^{(i)} \right)$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right)$$

3. The Simultaneous Parameter Update Rule

At each iteration t , both parameters must be updated **simultaneously** using the learning rate $\alpha > 0$:

$$w := w - \alpha \frac{\partial J}{\partial w} = w - \alpha \left[\frac{1}{m} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right) x^{(i)} \right]$$

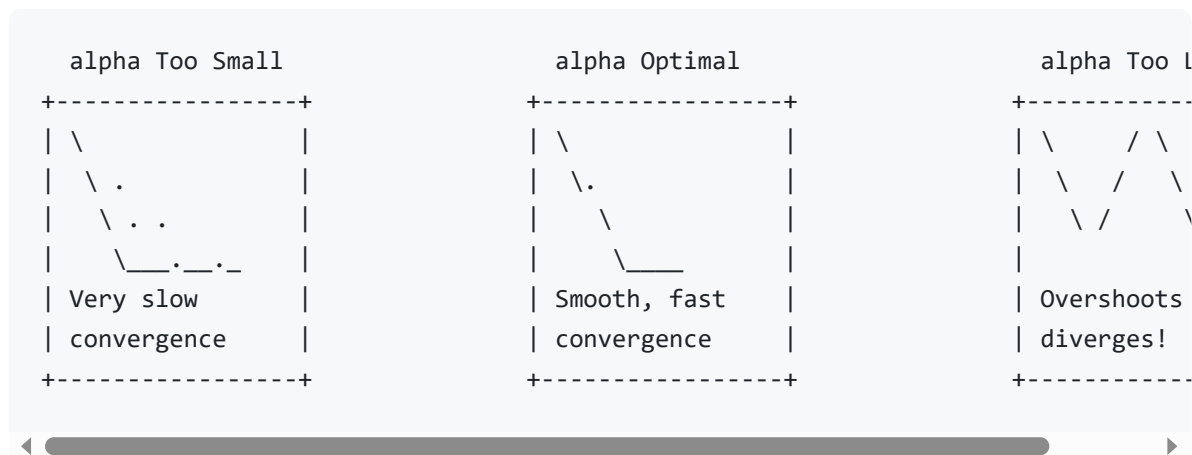
$$b := b - \alpha \frac{\partial J}{\partial b} = b - \alpha \left[\frac{1}{m} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right) \right]$$

Software Implementation Warning: Parameter updates must be computed using temporary intermediate variables to ensure that the updated w does not pollute the gradient computation of b :

```
temp_w = w - alpha * dj_dw
temp_b = b - alpha * dj_db
w = temp_w
b = temp_b
```

4. The Impact of Learning Rate α

The scalar hyperparameter α dictates the step size taken in the negative gradient direction:



1. **α Too Small:** Convergence is guaranteed on a convex surface, but requires tens of thousands of iterations, resulting in immense computational waste.
2. **α Balanced:** Rapid exponential decrease in cost J , converging smoothly to the minimum.
3. **α Too Large:** Steps overshoot the valley bottom. Error increases with each iteration ($J \rightarrow \infty$), causing numeric overflow (NaN).